ORIE 7190 (AI for Science, Fall 2026) HW 2

Out: Monday 2026-10-19 · **Due:** Friday 2026-10-30, 11:59pm, on Gradescope (course entry code `7BV55W`)

Covers the transformers sessions (Sep 21 – Sep 25), the language models lecture (Oct 2) and Reading 3. Question 3 covers the graph neural network and interatomic potential lectures (Sep 9 – Sep 18).

Ask questions on Ed. Hand in a python notebook, or a PDF with a code appendix. Upload your solutions on Gradescope.

Questions have two parts. - Part A — no AI assistance. Answer these *before* you write the code for Part B. These will be graded on your reasoning, not on whether you turned out to be right. - **Part B — AI encouraged.** Use whatever tools you like, as long as you can explain what your code is doing. **Note at the top which tools you used and where.**

Collaboration is fine. List who you worked with.

Dataset: SMILES strings from HW1's dataset

This code downloads and cleans data used in several questions in this HW.

```
import pandas as pd

URL = ("https://raw.githubusercontent.com/mcsorkun/AqSolDB/master/"
       "results/data_curated.csv")
df = pd.read_csv(URL)
df["SMILES"] = df["SMILES"].str.strip()        # 40 rows have a trailing space
df = df[df["SMILES"].str.len() >= 3].copy()    # drops 13 SMILES of one or two characters
```

Questions 1, 2 and 4 split each SMILES string into tokens with this function, based on Schwaller et al. (2019). The tokens are atom-level. Each atom, bond, ring-closure digit or parenthesis is one token, and a bracket atom such as `[Na+]` or `[nH]` is one token. Question 1 adds a token `$` at both ends of each molecule. The counts below are before that.

```
import re

ATOM_REGEX = re.compile(
    r"(\[.+?]|Br?|Cl?|N|O|S|P|F|I|b|c|n|o|s|p"
    r"|\(|\)|\.|=|#|-|\+|\\|\/|:|~|@|\?|>|\*|\%[0-9]{2}|[0-9])")

def tokenize(smi):
    tokens = ATOM_REGEX.findall(smi)
    assert "".join(tokens) == smi, smi    # the tokens rebuild the string
    return tokens
```

Check that you get these numbers before going on.

Molecules	9,969
Total tokens	298,536
Distinct tokens	223
V, distinct tokens plus `$`	224
Molecules of 63 tokens or fewer	9,479
Molecules of more than 63 tokens	490

Question 1 — Build an atom-level language model

Build a small transformer language model that learns to write SMILES one token at a time, by predicting each token from the ones before it. Train it on the dataset and sample molecules from it. Questions 2 and 4 reuse the trained model.

Model. We will use the model from the end of the transformers deck: a token embedding at the bottom, N transformer blocks, and a softmax over the vocabulary at the top. The vocabulary size (number of tokens) is $V = 224$. The context window is 64 tokens. The suggested model has about 859,000 parameters and trains for 3,000 steps. Details are at the end of the question.

Part A (no AI)

Answer these **before you train anything.** Item 2 needs a few lines of code on the data. No other code is needed.

1. **Predict the training loss at initialization.** Report your answer in nats per token, i.e., normalized by the sequence length. (A *nat* is a unit of log-loss using the natural logarithm, which is what `cross_entropy` returns by default.) Assume that the predicted next-token distribution at initialization is uniform over the vocabulary. Your answer should depend only on your vocabulary size V.
2. **Predict the training loss of a model that ignores context.** Take a model that learns the **overall frequency of each token** and nothing else. Its loss is the unigram entropy of the target tokens, $-\sum_c p_c \ln p_c$, where p_c is the fraction of target tokens that are c. Count each of the 9,479 kept molecules' tokens plus one closing `$`. Compute it from your data (a few lines of code), report it, and report the gap between it and your item 1 number. Then say what your model has to use to get below it.
3. **Validity.** An atom-level model has to satisfy **two bookkeeping constraints** to produce parseable SMILES, one involving parentheses and one involving digits. Name them, and say why each is harder for this architecture than predicting the next atom symbol. In the 9,479 molecules you keep, branches outnumber rings about two to one (27,026 against 12,610), but a `(` is matched by its `)` on average 4.5 tokens later, and a ring-closure digit by its partner 13.6 tokens later. Then predict **which of the two constraints your trained model will break more often**, and why.

Part B (AI encouraged)

4. Build the model. Report V and the total parameter count. Then measure the loss on one batch before any training step, twice: once with the output layer as PyTorch initializes it, and once after setting its weights and bias to zero with `nn.init.zeros_`. The zeroed version should match item 1 to four decimal places. If it does not, find out why before going on. A wrong number here usually means V, the output layer or the loss reduction is not what you think it is. The default version will come out higher. Use your answer to item 1 to explain why.
5. Train. **Plot train and held-out loss against training step, with your two reference lines from items 1 and 2 on the same axes.** Measure both in eval mode, the train loss on a fixed set of about 1,000 training molecules. Report the final values and say which reference lines your model got below.
6. Sample 500 molecules at $T = 1$ and report the fraction that are valid: the sample finished with `$`, is not empty, and `rdkit.Chem.MolFromSmiles(s) is not None`. Sort the invalid samples into four groups, checked in this order: unfinished, unclosed ring, unbalanced parentheses, and everything else. Report the count in each with an example, name the most common problem in "everything else", and say whether the ranking matches your prediction in item 3. RDKit's message for each rejected string names the problem. To see the messages in a notebook, call `rdkit.rdBase.LogToPythonStderr()` once.
7. Sample 500 again at temperatures 0.5, 1.0 and 1.5. For each, report the valid fraction **and the fraction of valid samples that are exact string copies of training molecules**, and read through 10 of the samples. Describe what changes as the temperature goes up. Then say how the size of the corpus affects the copy fraction.
8. Where did your Part A predictions miss?

Details for building the model

Use PyTorch's `nn.TransformerEncoderLayer` and `nn.TransformerEncoder` for the blocks, and write the rest yourself. PyTorch's defaults differ from the lecture in four places:

- **`batch_first=True`.** Otherwise PyTorch expects tensors shaped `(sequence, batch, feature)`.
- **`norm_first=True`.** This puts the LayerNorm before each sublayer, inside the residual, as in the deck. The default puts it after, as the 2017 paper did.
- **`dim_feedforward=4*d`.** The default is 2048 whatever d is, which at $d = 128$ gives 2.4 million parameters instead of about 859,000.
- **A causal mask.** Pass `mask=nn.Transformer.generate_square_subsequent_mask(n)` and `is_causal=True`, where `n` is the input length. Without the mask each position can see the token it is supposed to predict.

Also pass `norm=nn.LayerNorm(d)` to `nn.TransformerEncoder`. It normalizes the last block's output, as in GPT-2.

Dropout is on by default, with probability 0.1. Call `model.eval()` before you measure a loss for a plot, sample, or compute features, and `model.train()` before training again.

The two embeddings. The input to the first block is the sum of a learned **token** embedding, one row of length d per token, and a learned **position** embedding, one row per slot in the 64-slot context window. The output layer is a linear map from d to V logits.

Turning molecules into batches

- **Split each SMILES with `tokenize`** from the dataset section. `Cl` is one token, so is `[Na+]`, and `OC(=O)c1ccccc1Cl` is 15.
- **Add a token `$`** at both ends of every SMILES, so ethanol becomes `$CCO$`. The model reads `$CCO` and is trained to predict `CCO$`. Build the vocabulary from all 9,969 molecules, before dropping any, so that V **= 223 tokens + 1 = 224** matches the table. Thirty of the 223 tokens appear only in molecules you will drop.
- **One molecule per example**, not one long stream. Pad each input at the end to the context length with a separate pad token, and leave pad positions out of the loss with `cross_entropy(..., ignore_index=pad_id)`. The pad token gets a row in the token embedding but no output in the output layer, so it is not counted in V.
- **Context length 64.** The input is the SMILES plus the leading `$`, so this fits molecules of up to 63 tokens. **Drop the longer ones** (there are 490). Report how many you dropped.
- **Hold out 10% of what remains** as a validation set, with a stated seed. Train on the other 90%.

Training and sampling

We suggest $d = 128$, 4 heads, 4 blocks, batch size 64, AdamW with learning rate 3×10^{-4}, and 3,000 steps. You can change any of these. Report what you used. A 3,000-step run took 14.5 minutes on a busy laptop CPU (Apple M3). On a Mac, `device="mps"` is faster.

Save the model at steps 0, 300, 1,000 and 3,000, or at the end of your run if you train for a different number of steps. Questions 2 and 4 use these checkpoints.

To sample, start from `$`. Feed the model what it has produced so far, take the logits at the last position, divide them by the temperature T, apply softmax, draw one token, and append it. Stop when it draws `$` or the input reaches 64 tokens. A sample that reaches 64 tokens without drawing `$` is unfinished.

Question 2 — What does pretraining help predict?

The transformers deck ended with the claim that "the prediction is a pretext — the representation is the product." You train a network to predict the next token, throw away the prediction head, and keep $f(x)$, the

vector the network computes just below the head. Then you fit a cheap model, such as logistic regression, from $f(x)$ to the few labels you have. This cheap model is a *probe*, like the one you fit on an MLP's representation in HW1 Question 4.

Here you test the claim on your Question 1 model, which was trained only to predict the next token. To tell whether pretraining is what helps, Li et al. (Reading 3) compared pretrained protein language models against three baselines, each of which removes one thing. They call pretraining **aligned** with a task when the pretrained model beats all three baselines *and* its performance improves with scale.

The two targets

Both are columns of the AqSolDB file.

- **Solubility**, HW1's target. It is measured in the lab, and it depends on how the whole molecule interacts with water: its size, how oily it is, and how much of its surface can hydrogen-bond. The probe's target is `Solubility > median`.
- **Saturated rings**, the column `NumSaturatedRings`: the number of rings in which every bond is a single bond, computed by RDKit from the structure. Cyclohexane `C1CCCCC1`, piperidine `C1CCNCC1` and cyclohexanone `O=C1CCCCC1` each have one. The C=O of cyclohexanone sits outside the ring. Benzene `c1ccccc1` and cyclohexene `C1=CCCCC1` have none. Most molecules here have no saturated ring, so the probe's target is whether a molecule has **at least one**, which is true for 17.1% of them.

The baselines

Baseline	What it removes	Here
One-hot	the network	the count of each of the 223 tokens in the SMILES string, divided by its number of tokens
Random weights	training	your architecture at initialization, 3 seeds
Shuffled weights ("stat transfer")	where each trained weight sits, keeping the values	your trained model with every weight matrix shuffled, 3 seeds

Shuffling the weights

The shuffled baseline keeps every number in your trained model but moves each one to a random place within its own weight matrix. The weight matrices are the token and position embeddings, the output layer, and in each block the attention matrices and the feed-forward net's two matrices. In attention, W_Q, W_K and W_V turn each position's vector into a query, a key and a value, and W_O combines the heads. PyTorch stores W_Q, W_K and W_V stacked in one matrix, `in_proj_weight`, and shuffles it as one matrix. Leave the biases and the LayerNorm parameters alone. Each matrix keeps its mean and spread, but the arrangement learned in training is gone. The two-dimensional parameters are exactly the weight matrices, so this does it:

```
g = torch.Generator().manual_seed(0)
with torch.no_grad():
    for name, p in model.named_parameters():
        if p.dim() == 2:      # weight matrices only: skips biases and LayerNorm
            p.copy_(p.flatten()[torch.randperm(p.numel(), generator=g)].view_as(p))
```

Setup

Fit the probes on the three baselines and on your model's representation. For solubility, also fit them on HW1's 17 descriptors. Leave the descriptors out for saturated rings, because they include `NumSaturatedRings` itself. **Use the 9,479 molecules that Question 1 kept**, the ones of 63 tokens or fewer. The longer ones do not fit in your model's context window.

```
df2 = df[df["SMILES"].map(lambda s: len(tokenize(s)) <= 63)].copy()
y_sol = (df2["Solubility"] > df2["Solubility"].median()).astype(int).values
y_ring = (df2["NumSaturatedRings"] >= 1).astype(int).values
```

Use one split for everything in this question, `train_test_split(..., test_size=2000, stratify=y_sol, random_state=0)`. Refit the descriptor baseline on this split.

Standardize every feature set and use `LogisticRegression(C=1e6, max_iter=20000)`, as in HW1. Also fit `RidgeCV(alphas=np.logspace(-3, 3, 13))` to the continuous values, the `Solubility` column (already a base-10 log) and `NumSaturatedRings`, and report test R^2.

Part A (no AI)

Answer these **before running anything.**

1. **Where could the information come from?** Your model never saw either label. For solubility, name one feature of a SMILES string that a next-token predictor has to represent well and that also matters for dissolving. For saturated rings, say what the model would have to keep track of while reading a SMILES string to know whether a ring it has just closed is saturated.
2. **Which target will pretraining help more?** For each target, predict the test accuracy of your trained LM's representation and of the random-weights baseline. Say which target shows the larger gap and why, using item 1 and your answer to Question 1 item 3.
3. **Predict whether pretraining is aligned** with each property, by Li et al.'s definition. Then predict whether your language model beats the descriptors on solubility. Chemists chose those 17 numbers *because* they predict solubility, and `MolLogP` is close to the answer by construction. **Say what would have to change for your answer to flip.**

Part B (AI encouraged)

4. Build the representation. With the model in eval mode, feed each molecule in as the model saw it in training, `$` followed by the SMILES, run it through every block and the final LayerNorm but not the output layer, and **average the resulting vectors over the non-pad positions.** Then fit the probes on the one split and report one table: a row per feature set, and test accuracy and test R^2 for each target, with the mean and range over the 3 seeds for the random- and shuffled-weights baselines. Report each target's majority-class rate as well. **Which differences are larger than the seed range, and for which target?**
5. **Does it scale?** Use the four checkpoints from Question 1. For each, record its held-out next-token loss and the probe's test R^2 for both targets. (Your step-0 checkpoint is one draw of the random-weights baseline.) Plot R^2 against held-out loss and **compute the Spearman rank correlation between R^2 and minus the held-out loss**, for each target. Use R^2 rather than accuracy here, because accuracy on the ring target starts near its ceiling. Li et al. call a task *scaling* when that correlation is above 0.9. Report yours, and say whether four points are enough to take it seriously.
6. Where did your Part A predictions miss? Then answer, with your numbers: **for which property is pretraining aligned, by Li et al.'s definition, and what is different about that property?**

Question 3 — A machine-learned potential where you know the truth

Fit a machine-learned potential to configurations of 13 atoms whose energy is given by the formula,

$$E = \sum_{i<j} 4\left(r_{ij}^{-12} - r_{ij}^{-6}\right), \qquad r_{ij} = \|\mathbf{R}_i - \mathbf{R}_j\|,$$

This is the *Lennard-Jones potential*, in units where its two constants ε and σ are 1. Having an exact formula will let us check claims about fit. Note that the assumption made by MLIPs about locality is exactly true in

this model, because the whole cluster fits inside one cutoff. Details on the cluster, data, models and training are at the end of the question.

You will compare four models with about 5,000 parameters each:

Model	Input	Energy
1. Raw MLP	the 39 coordinates	MLP output
2. Sorted-distance MLP	the 78 pair distances, sorted	MLP output
3. Atom-sum MLP	each atom's 12 distances to the others, sorted	sum over atoms of one shared MLP
4. Pair GNN	the 78 pair distances, as edges between all 13 atoms	message passing, then a sum over atoms

Part A (no AI)

Answer these **before running anything.**

1. **Energy-only against energy+force.** You will fit models 3 and 4 on energies alone and on energies plus forces. Predict how many times more configurations energy-only training needs to reach the same **test energy error**.
2. **Four models at matched size.** Rank the four models by test energy error after energy+force training on 200 configurations. For each, say which of translation, rotation and atom relabeling it gets **for free** and which it must **learn from data**. The Lennard-Jones energy is a sum over pairs of one function of distance. Which model's structure has that form, and why should that change how much data it needs? Use the SchNet slide in the MLIP deck and the message-passing slides in the GNN deck.
3. **Compression.** You will train near the minimum (noise $s = 0.05$) and test on the relaxed cluster squeezed toward its center so that every distance shrinks by about 10%. Will the models predict energies too high or too low? Justify it from the training distribution. Then name the slide in the MLIP deck you would be reproducing.

Part B (AI encouraged)

4. Write E and the analytic force. On a few random configurations, compare your analytic force with PyTorch autograd and with a central finite difference, in float64, and report the largest absolute discrepancy for each.
5. **Learning curves.** Fit models 3 and 4 at $n = 50$, 200 and 800, once energy-only and once energy+force. For each model, plot test energy and force error against n for both losses, on log-log axes, and estimate the ratio of n at matched test energy error. If the curves don't reach the same error within this range, give a bound on the ratio instead. Compare with item 1. Then fit models 1 and 2 with energy+force at $n = 200$, so you have all four at $n = 200$. Does their ranking by test energy error match item 2?
   - **Rotation test.** Report each model's mean absolute change in predicted energy between a test configuration and its rotated copy, next to its test energy error. Explain model 1's number.
6. **Compression.** Take the four energy+force models trained on $n = 200$ in item 5. Plot true and predicted energy against c over both compression sweeps. Report the sign and size of the error, and whether it matches item 3. Then add 20 compressed configurations to the training set, refit and replot. How much of the gap closes? Connect this to the "Both sides are right" slide.
7. Where did your Part A predictions miss?

The relaxed cluster

Put one atom at the origin and twelve at $(0, \pm 1, \pm\varphi)$ and their cyclic permutations, $\varphi = (1+\sqrt{5})/2$, rescaled to distance 1.1 from the origin. Relax with `scipy.optimize.minimize(..., jac=True, method="L-BFGS-B")`, with your function returning the energy and its gradient, which is minus your analytic force. Then center the cluster. **Check that you get $E = -44.3268$ before going on.**

Generating configurations

Add independent Gaussian noise of standard deviation s to every coordinate of the relaxed cluster. Train at $s = 0.05$ on $n = 50$, 200 or 800 configurations. Test on 1,000 fresh configurations at $s = 0.05$ and on a randomly rotated copy (`scipy.spatial.transform.Rotation.random`). Test energy error is the mean absolute error of $\hat{E}$ over the test configurations, and force error is the mean absolute error over all force components.

Don't rotate, translate or relabel the training configurations. Model 1 then never sees a rotated cluster, which is why the rotation test means something.

The compression sweep scales the relaxed cluster about its centroid by $c \in \{1.00, 0.97, 0.94, 0.91, 0.88, 0.85\}$. Make one set without noise and one with 100 configurations at each c with $s = 0.05$. For item 6's refit, draw the 20 extra configurations with c spread evenly over 0.85 to 0.97, each with $s = 0.05$ noise.

Building the four models

Model 3 takes the per-atom sum from the MLIP deck's hand-designed descriptor slide, with sorted distances as the descriptor. Model 4 runs two rounds of the SchNet slide's update, every atom starting from one shared learned vector, then $\hat{E} = \sum_i \mathrm{MLP}(\mathbf{h}_i)$. Feed the filter W a vector of Gaussian radial basis functions of r_{ij}.

Give every model about 5,000 parameters, within 10%, and report the counts. Standardize the inputs of models 1-3 (coordinates or distances) by their mean and standard deviation over the training set. Without this, model 1 barely trains.

Use a smooth activation such as SiLU, because forces are gradients of the energy.

Compute forces with `torch.autograd.grad(E.sum(), pos, create_graph=True)`, so that the force loss can be backpropagated.

Training

With σ_E the standard deviation of the training energies and σ_F that of the training force components, minimize

$$\mathcal{L} = \mathrm{mean}\left[(\hat{E} - E)^2\right]/\sigma_E^2 + w\ \mathrm{mean}\left[(\hat{F} - F)^2\right]/\sigma_F^2,$$

the first mean over configurations, the second over every force component. This is the loss on the MLIP deck's "Unobserved per-atom energies, supervised forces" slide, normalized, with w in place of λ. Setting $w = 0$ gives energy-only training. If your network outputs a normalized energy, convert it back to $\hat{E}$ before differentiating.

We suggest full-batch Adam for 5,000 steps, with the learning rate starting at 10^{-2} and decaying along a cosine to 1% of that, and $w = 1$. Model 4 costs several times more per step than the others. On a laptop CPU, fits range from under a minute (models 1-3 at $n = 50$) to tens of minutes (model 4 at $n = 800$ with forces).

Question 4 — Do element representations follow the periodic table's groups?

The columns of the periodic table are called **groups**. Elements in the same group usually form the same number of bonds and tend to behave alike. Your Question 1 model was never told which group an element is in. It saw only SMILES text.

The hypothesis is that two elements in the same group have representations that are closer together than two elements in different groups.

To test this hypothesis, you will run the following procedure on several sets of vectors, where each vector represents an element.

- Compute the **cosine distance**, $1 - u \cdot v/(\|u\| \, \|v\|)$, between every pair of elements.
- Split the pairs into **same-group** and **different-group** pairs, and **plot the two sets side by side**, one dot per pair, with the mean of each set marked.
- Summarize with one number, A. Take every combination of one same-group pair and one different-group pair, and report the fraction in which the same-group pair is closer. $A = 0.5$ means no group structure, and $A = 1$ means every same-group pair is closer than every different-group pair.

No training in this question.

The language model's vectors

Use these 13 elements. They are the elements that are (1) written at least 30 times in the 9,479 molecules Question 1 kept, not counting lowercase (aromatic) atoms, and (2) share a group with at least one other element on the list. Hydrogen also meets both conditions but is left out, because SMILES leaves most hydrogens implicit and 57 of its 64 written atoms are `[H+]` ions.

Element	Group	How it is written, with counts
C	14	`C` 70,941, `[C@@H]` 573, `[C@H]` 541, 8 other forms 310
Si	14	`[Si]` 120, 3 other forms 5
N	15	`N` 8,151, `[N+]` 796, 9 other forms 237
P	15	`[P]` 375, `P` 37, 3 other forms 24
O	16	`O` 21,621, `[O-]` 1,640, `[O--]` 153, 3 other forms 39
S	16	`S` 1,196, `[S]` 799, 5 other forms 46
F	17	`F` 1,152, `[F-]` 33, `[F]` 3
Cl	17	`Cl` 3,694, `[Cl-]` 172, `[Cl]` 7
Br	17	`Br` 474, `[Br-]` 21
I	17	`I` 127, `[I-]` 13, `[I]` 1
Li	1	`[Li+]` 35, `[Li]` 2
Na	1	`[Na+]` 275, `[Na]` 3
K	1	`[K+]` 56, `[K]` 4

Each form is a separate token with its own row in the token embedding, so every element on the list has at least one row and some have several. Rather than choose among those rows, use the model's hidden vectors. With the model in eval mode, feed each of the 9,479 molecules in as in training, `$` followed by the SMILES, and take the vector at every position after the final LayerNorm, as in Question 2 item 4 but before averaging. For each element, **average the vectors over every position that holds one of its tokens**. Skip lowercase (aromatic) atoms such as `c`, `n` and `[nH]`. Nearly all of them are other tokens for C, N, O and S. An atom token's element is its first capital letter plus an optional lowercase letter: `re.match(r"\[?\d*([A-Z][a-z]?)", t)` gives `Cl` for `Cl`, `Na` for `[Na+]` and `C` for `[C@@H]`, and no match for aromatic atoms or for tokens that are not atoms. Token `i` of a SMILES is at input position `i + 1`, because of the leading `$`.

Do this for two of your Question 1 checkpoints: step 0, which is an untrained network, and your final one.

Part A (no AI)

Answer these **before running anything.**

1. **The language model.** To predict the next token, the model has to learn what can come before and after each atom. Using the last column of the element table, predict for each of the five groups whether its members will come out close together, and say why. Which group do you expect to be clearest? Is there a group whose members you expect to be far apart?

2. Predict A for the 13 elements at step 0 and at your final checkpoint. Is the step-0 value necessarily 0.5? Say why or why not.

Part B (AI encouraged)

3. **The language model.** Build the 13 averaged vectors from the step-0 checkpoint and from your final one. For each, make the plot and report A. Then compute A **for each group separately**, using only that group's same-group pairs against all the different-group pairs. Which groups come out close together? For any group that does not, list its members' two nearest elements and explain them using the element table.
4. **A baseline with no network.** For each of the 13 elements, count the token just after each of its atoms and the token just before, using `$` for the start and end of the string. Normalize each of the two counts to sum to 1 and join them into one vector. Use the same atoms as in item 3. Repeat item 3's plot, A and per-group A. What did the trained network add?
5. Where did your Part A predictions miss? This part carries the most weight in the question.
